C# Programlama Dili Kapsamlı Ders & Tekrar Notları

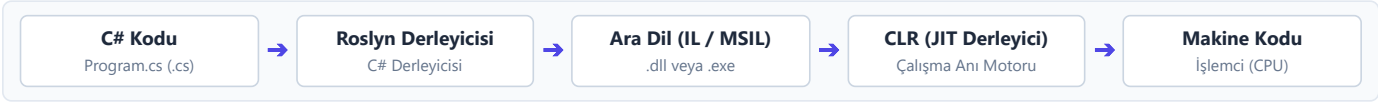
Temel Çalışma Mantığından Modern Nesne Yönelimli Tasarıma (OOP) Uygulamalı Rehber

📚 **Kapsam:** Bellek Mimarisi, Namespace & using, get/set Kapsülleme, ref/out, OOP, using Kaynak Yönetimi, LINQ

💡 **Hedef Kitle:** Öğrenciler ve Başlangıç-Orta Düzey Geliştiriciler

1. C# Çalışma Mantığı ve Derleme Mimarisi

C# kaynak kodu doğrudan bilgisayar işlemcisinin (CPU) anlayacağı 0 ve 1'lerden oluşan makine koduna dönüştürülmez. İki aşamalı, güvenli ve platformdan bağımsız bir derleme sürecinden geçer:



- **Roslyn Derleyicisi:** Yazdığımız C# kodunu denetler; syntax hatası yoksa ara dile (IL - Intermediate Language) dönüştürür.
- **CLR (Common Language Runtime):** .NET programlarının sanal motorudur. Belleği yönetir, güvenliği sağlar ve istisnaları yakalar.
- **JIT (Just-In-Time) Derleyici:** Program çalıştığı anda (çalışma anında - runtime), IL kodunu o anki bilgisayarın işlemcisine özel makine koduna çevirir.
- **Garbage Collector (GC - Çöp Toplayıcı):** Program çalışırken artık kullanılmayan, sahipsiz kalan nesneleri Heap belleğinden otomatik olarak silip belleği temizler. Geliştiricinin C++'taki gibi manuel bellek silmesine gerek kalmaz.

2. Bellek Mimarisi: Stack vs. Heap (Hangi Veri Nerede Yaşar?)

Bilgisayar belleği (RAM) programlama dillerinde iki ana bölgeye ayrılır. C#'taki tüm veri tipleri bu iki bölgeden birinde saklanır:

Özellik	Stack (Yığın) — Değer Tipleri	Heap (Öbek) — Referans Tipleri
Tipler	`int`, `decimal`, `double`, `bool`, `char`, `struct`, `enum`, `DateTime`	`class`, `string`, `List<T>`, diziler (`array`), `interface`
Çalışma Hızı	Çok hızlıdır. Boyutları derleme anında bellidir.	Stack'e göre daha yavaştır; dinamik olarak büyüyebilir.
Veri Saklama	Verinin doğrudan **kendisi** Stack'te saklanır.	Nesnenin **kendisi Heap'tedir**; Stack'te ise o nesnenin Heap'teki adresi (işaretçisi / pointer) durur.
Kopyalama Davranışı	**Değer kopyalanır.** Bir değişkenin değişmesi diğerini asla etkilemez.	**Adres kopyalanır.** İki değişken aynı nesneyi gösterir; birindeki değişiklik diğerine yansır.
Varsayılan Değer	Sayılar için `0`, boolean için `false`	`null` (Hiçbir adresi göstermiyor demektir)

Değer Tipi Kopyalama (Stack)

```
int a = 10;
int b = a;  // Değerin kopyası alındı
b = 20;     // b değişti

// Sonuç: a hala 10'dur! (Birbirinden bağımsız)
```

Referans Tipi Kopyalama (Heap)

```
Calisan c1 = new Calisan(); c1.Ad = "Ali";
Calisan c2 = c1; // Aynı bellek adresi paylaşıldı
c2.Ad = "Veli";

// Sonuç: c1.Ad da "Veli" oldu! (Aynı nesne)
```

3. Field (Alan) ve Property (Özellik) Ayrımı

Bir C# sınıfında veri saklamanın iki yolu vardır. Yeni başlayanların en sık karıştırdığı konulardan biri budur:

- **Field (Alan / Düz Değişken):** Sınıfın ham verisidir. Güvenlik filtresi yoktur. Dışarıya açık bırakılması önerilmez.
- **Property (Özellik):** Field'ın önüne konulan akıllı güvenlik kapısıdır. Dışarıdan değişken gibi görünür ama arka planda `get` ve `set` fonksiyonları çalıştırır.

1. Field (Düz Alan - Kontrolsüz)

```
public class Urun
{
    // Düz değişken: Herkes istediğini yazar!
    public decimal Fiyat;
}

// Kullanımı:
Urun u = new Urun();
u.Fiyat = -500; // Negatif fiyat engellenemez!
```

2. Property (Akıllı Özellik - Kontrollü)

```
public class Urun
{
    private decimal _fiyat; // Gizli field
    public decimal Fiyat    // Güvenli property
    {
        get { return _fiyat; }
        set { _fiyat = value < 0 ? 0 : value; }
    }
}
```

⚠️ **WinForms Geliştiricileri İçin Hayati Püf Noktası:**
Formunuzdaki bir `DataGridView` tablosuna `dataGridView1.DataSource = calisanListesi;` dediğinizde, ekranda **sadece Property olan üyeler sütun olarak listelenir!** Düz Field olarak tanımladığınız değişkenler ekranda asla görünmez. Bu yüzden modellerinizde her zaman property kullanmalısınız.

4. Kapsülleme (Encapsulation), get/set ve `value` Mantığı

Kapsülleme, bir nesnenin iç verilerini dış müdahalelerden koruma sanatıdır. PHP'deki `getAd()` ve `setAd()` fonksiyonlarının C#'taki karşılığıdır:

Klasik Açık Yazım (Full Property) ve `value` Kelimesi

```
public class Calisan
{
    private string _ad; // Backing Field: Veriyi bellekte saklayan gizli kasa

    public string Ad    // Dış dünyaya açılan kapı
    {
        get { return _ad; }        // Değer okunurken çalışır
        set { _ad = value.Trim(); } // Değer atanırken çalışır ('value' atanan yeni veridir)
    }
}
```

💡 **Otomatik Özellik Kısayolu (Auto-Property):**
Eğer özel bir doğrulama şartı yoksa C# derleyicisi tek satırda yazmamıza izin verir: `public string Ad { get; set; } = "";`
Buradaki `= "";` başlangıç atamasıdır. Nesne oluşturulduğunda alanın `null` kalmasını engeller ve programın çökmesini önler.

5. Erişim Belirleyiciler (Access Modifiers)

Belirleyici	Erişim Kapsamı	Günlük Hayat Benzetmesi
`public`	Dünyadaki herkese açık. Tüm projeler ve sınıflar erişebilir.	Şirketin **ana giriş kapısı** (Herkes girebilir).
`internal`	**Sadece aynı proje (aynı DLL / EXE)** içindekiler erişebilir. Dış projeler göremez.	Şirketin **personel yemekhanesi** (Sadece çalışanlar).
`private`	**Yalnızca o sınıfın içi** erişebilir. Aynı projedeki diğer sınıflar bile göremez.	Personelin **kilitli şahsi çekmecesi**.
`protected`	Sadece tanımlandığı sınıf ve o sınıftan **miras alan alt sınıflar** erişebilir.	Bir ailenin **miras sandığı** (Sadece çocuklar).

6. Namespace (İsim Uzayı) ve `using` Mimarisi

Büyük projelerde onlarca klasör ve yüzlerce sınıf (class) bulunur. **Namespace (İsim Uzayı)**, aynı isimdeki sınıfların birbiriyle çakışmasını önleyen ve projeyi mantıksal bölümlere ayıran **dijital soyadı / posta adresi** sistemidir.

> 🌭 **Günlük Hayat ve PHP Benzetmesi:**
> Bir okulda üç tane "Ali" isimli öğrenci olduğunu düşünün. Öğretmen sadece "Ali ayağa kalk" derse karmaşa çıkar. Ancak **"Demir ailesinden Ali"** veya **"Yılmaz ailesinden Ali"** dendiğinde kim olduğu kesinleşir. İşte `namespace Teknik_Servis_Yönetim.Models` bir ailenin soyadı gibidir.
> **PHP Karşılaştırması:** PHP'deki `namespace App\Models;` ve `use App\Models\Calisan;` yapısının C#'taki birebir karşılığıdır: `namespace Teknik_Servis_Yönetim.Models;` ve `using Teknik_Servis_Yönetim.Models;`.

A) "Dosyayı Yeni Klasöre Taşıdım, Kod Neden Hata Vermedi / `using` İstemedik?"

Yeni başlayanların en sık karşılaştığı kafa karışıklığı şudur: *"Ben bir UserControl dosyasını UserControls klasörüne taşıdım ama ana form hala using eklemeden bu sınıfı gördü, neden?"*

> ⚠️ **Kritik Kural: Fiziksel Klasör ≠ Mantıksal Namespace!**
> - C#'ta bir dosyayı Windows Gezgini'nde veya Visual Studio'da yeni bir klasöre taşımak, **o dosyanın namespace'ini otomatik olarak DEĞİŞTİRMEZ!**
> - Dosyanın en üstünde hala `namespace Teknik_Servis_Yönetim` yazıyorsa, dosya hangi klasörün içinde olursa olsun projenin ana kök çatısındadır.
> - Diğer sınıfların `using` istemesi için, taşınan dosyanın içindeki namespace satırını da elle `namespace Teknik_Servis_Yönetim.UserControls` olarak güncellemeniz gerekir.

1. Uzun Yol (Tam Adres / Fully Qualified)

Namespace köprüsü kurmadan doğrudan soyadıyla çağırmak:

```
// Her yerde uzun uzun tam adres yazılır:
Teknik_Servis_Yönetim.UserControls.UcCalisanListesi uc =
    new Teknik_Servis_Yönetim.UserControls.UcCalisanListesi();
```

2. Kısa Yol (`using` ile Köprü Kurma)

Dosyanın en tepesine `using` ekleyerek kestirme yol açmak:

```
// Dosyanın en başında 1 defa köprü açılır:
using Teknik_Servis_Yönetim.UserControls;

// Artık kod içinde tek isimle kullanılır:
UcCalisanListesi uc = new UcCalisanListesi();
```

B) WinForms Geliştiricileri İçin Hayati Tehlike: `partial class` ve `.Designer.cs`

> 🚨 **Dikkat: Bir Form/UserControl İki Parçadan Oluşur!**
> Windows Forms'ta örneğin `UcCalisanListesi` kontrolü fiziki olarak iki dosyadır:
> 1. `UcCalisanListesi.cs` (Sizin yazdığınız iş kodları)
> 2. `UcCalisanListesi.Designer.cs` (Visual Studio'nun butonları ve kutuları çizdiği otomatik kodlar)
>
> Bu iki dosya `partial class` (parçalı sınıf) anahtar kelimesiyle birbirine bağlanır. **Eğer ana `.cs` dosyasının namespace'ini değiştirip `.Designer.cs` dosyasını eski haliyle bırakırsanız;** derleyici bu iki yarının aynı sınıf olduğunu anlayamaz, `InitializeComponent()` kaybolur ve derleme hataları oluşur. Bir formu taşırken her iki dosyanın da namespace'ini birebir aynı yapmalısınız!

> 💡 **Modern C# 10 Yeniliği: File-Scoped Namespace (Dosya Kapsamlı Namespace):**
> Eski C#'ta tüm kodlar `namespace X { ... }` süslü parantezleri içine alınırdı ve 1 tab sağa kayardı. Modern C#'ta dosyanın en üstüne noktalı virgülle `namespace Teknik_Servis_Yönetim.Models;` yazmak yeterlidir; fazladan süslü parantez karmaşasını ortadan kaldırır.

7. Modern C# Sözdizimi Şekerleri (Syntactic Sugar)

C#'ta kodu kısaltmak ve okunabilirliği artırmak için kullanılan pratik yazım kuralları:

A) `=>` (Fat Arrow / Expression-Bodied Members) Kısaltması

Tek satırlık metot veya özelliklerde süslü parantez ({ }) ve `return` yazma zahmetini ortadan kaldırır:

Geleneksel Yazım (Klasik Açık Yol)

```
public bool GecerliMi()
{
    if (Hatalar.Count == 0)
        return true;
    else
        return false;
}

public string TamAd
{
    get { return Ad + " " + Soyad; }
}
```

Modern Kısayol (`=>` ile)

```
// Tek satırlık metot:
public bool GecerliMi() => Hatalar.Count == 0;


// Tek satırlık salt-okunur Property:
public string TamAd => Ad + " " + Soyad;
```

B) Sayısal Ekler (Suffixes) — `1200m` Kodundaki `m` Harfi Nedir?

C#'ta yazılan ondalıklı sayılar varsayılan olarak `double` kabul edilir. Muhasebe ve para hesaplarında yuvarlama hatası yapmayan yüksek duyarlıklı **`decimal`** tipini belirtmek için sayının sonuna mutlaka **`m`** (Money) eklenmelidir:

Ek	Veri Tipi	Örnek Kod	Kullanım Amacı
`m` / `M`	`decimal` (128-bit)	`decimal maas = 25000.50m;`	Para birimleri, maaş, muhasebe ve finans hesaplamaları.
`f` / `F`	`float` (32-bit)	`float oran = 3.14f;`	Grafik, oyun motorları ve düşük bellekli kayan nokta sayıları.
`d` / `D`	`double` (64-bit)	`double pi = 3.14159;`	Bilimsel ve genel mühendislik hesaplamaları (Varsayılandır).
`L`	`long` (64-bit)	`long tcNo = 12345678901L;`	Standart int (2 milyar) sınırını aşan büyük tam sayılar.

C) String Interpolation (`$""`) ile Temiz Metin Formatlama

```
// Eski Yol (+ ile birleştirme):
string mesaj = "Çalışan: " + ad + " " + soyad + ", Maaş: " + maas.ToString();

// Modern Yol ($"" ile doğrudan içine yazma):
string mesaj = $"Çalışan: {ad} {soyad}, Maaş: {maas:C2}, Tarih: {DateTime.Today:dd.MM.yyyy}";
// ':C2' parayı otomatik olarak TL (₺) formatında yazar. ':dd.MM.yyyy' tarihi gün.ay.yıl yapar.
```

8. Metot Parametreleri: Normal, `ref` ve `out` Farkı

Normal Aktarım (Değer Kopyalanır)

```
void Arttir(int sayi) { sayi += 5; }

int x = 10;
Arttir(x);
// x hala 10'dur! Kopya değişti, orijinal kaldı.
```

`out` Parametresi (Çıktı Kanalı)

```
bool Bol(int a, int b, out int bolum)
{
    bolum = a / b; // Değer atamak ZORUNLUDUR
    return true;
}
Bol(10, 2, out int sonuc); // sonuc = 5 olur!
```

> 💡 **`TryParse` Güvenliği:** TextBox'tan sayı alırken `Convert.ToInt32()` kullanırsanız kullanıcı harf yazdığında program çöker. Bunun yerine çökme riskini sıfırlayan `int.TryParse(txt.Text, out int sayi)` kullanın. Sayı geçerliyse `true` döner ve sayıyı `out` değişkenine doldurur!

9. Nesne Yönelimli Programlamanın (OOP) 4 Temel Direği

1. Kapsülleme (Encapsulation)

Veriyi gizleyip kontrollü property ile sunma.

```
public class Hesap {
    private decimal _bakiye;
    public decimal Bakiye => _bakiye;
}
```

2. Kalıtım (Inheritance)

Ortak özellikleri üst sınıftan miras alma.

```
public class Personel { public string Ad { get; set; } }
public class Teknisyen : Personel { public string Brans { get; set; } }
```

3. Çok Biçimlilik (Polymorphism)

Üst sınıftaki davranışı alt sınıfta ezme (`override`).

```
public class Cihaz {
    public virtual void Calis() { }
}
public class Yazici : Cihaz {
    public override void Calis() => Console.WriteLine("Yazdırıyor...");
}
```

4. Soyutlama (Abstraction & Interface)

Yapılacak işlerin şablonunu/sözleşmesini belirleme.

```
public interface ICalisanRepository {
    bool Ekle(Calisan calisan);
}
public class CalisanRepository : ICalisanRepository {
    public bool Ekle(Calisan c) { /* SQL */ return true; }
}
```

10. Generic Koleksiyonlar (`List<T>`) ve İstisna Yönetimi

`List<T>` (Tip Güvenli Dinamik Liste)

Eski diziler gibi sabit boyutlu değildir; dinamik büyür:

```
List<Calisan> liste = new List<Calisan>();
liste.Add(new Calisan { Ad = "Ahmet" });
liste.RemoveAt(0); // 0. elemanı sil
int toplam = liste.Count; // Eleman sayısı
```

`try-catch-finally` ve `using` Güvencesi

Programın çalışma anında çökmesini engeller:

```
try {
    conn.Open(); // Riskli işlem
} catch (Exception ex) {
    MessageBox.Show($"Hata: {ex.Message}");
} finally {
    conn.Close(); // Her durumda çalışır
}
```

11. Masaüstü Uygulamalarında Oturum Yönetimi (Session)

Web projelerinde PHP veya ASP.NET'te oturum bilgisi `$_SESSION` dizisinde saklanır. Windows Forms masaüstü uygulamalarında ise oturum bilgisi **`static class Oturum`** mimarisi ile yönetilir:

Temiz ve Güvenli Oturum Mimarisi:

```
public static class Oturum
{
    // Giriş yapan çalışanın tüm nesnesi burada saklanır:
    public static Calisan? AktifCalisan { get; set; }

    // Oturum açık mı kontrolü (Salt-okunur lambda property):
    public static bool GirisYapildiMi => AktifCalisan != null;

    // Güvenli çıkış yapma:
    public static void CikisYap() => AktifCalisan = null;
}
```

1. Giriş Formunda (Login):

```
// Başarılı girişten sonra:
Calisan bulunan = _service.GirisYap(tc, sifre);
Oturum.AktifCalisan = bulunan;

// Ana formu aç
new form_anasayfa().Show();
```

2. Herhangi Bir Form / UserControl İçinde:

```
// Artık projenin her yerinden doğrudan erişilebilir:
lbl_kullanici.Text = $"Hoş geldin, {Oturum.AktifCalisan?.Ad}";

if (Oturum.AktifCalisan?.Yetki != "Yönetici")
    btn_calisan_sil.Enabled = false; // Yetki kısıtlama
```

12. `using` Deyimi ve Kaynak Yönetimi (Resource Management)

C#'ta `using` anahtar kelimesi iki tamamen farklı amaçla kullanılır. Bu ayrımı bilmek bellek sızıntılarını ve veritabanı kilitlenmelerini önlemenin anahtarıdır:

Kullanım Şekli	Kod Örneği	Görevi ve Çalışma Mantığı
1. using Direktifi (Namespace Köprüsü)	`using System.Data;` `using Teknik_Servis.Models;`	Dosyanın en tepesine yazılır. Başka bir isim uzayındaki sınıfları projeye dahil eder, uzun uzun adres yazmayı engeller.
2. using Deyimi (Kaynak Temizleme)	`using (var conn = ...) { }` `using var cmd = ...;`	Metot içinde yazılır. İşi biten veritabanı bağlantısı veya dosya akışı gibi harici sistem kaynaklarını anında bellekten siler (`Dispose` eder).

> ⚠️ **Neden `conn.Close()` Tek Başına Yetersizdir? (Bağlantı Havuzunun Tükenmesi):**
> Veritabanına `conn.Open()` dedikten sonra aradaki SQL kodunda beklenmeyen bir hata (hata fırlatma / Exception) olursa, aşağıdaki `conn.Close()` satırına **asla ulaşılamaz!** Bağlantı MySQL sunucusunda açık asılı kalır. Birkaç kullanıcı işlem yaptığında MySQL "*Too many connections*" (Çok fazla bağlantı) hatası verir ve tüm program kilitlenir!

Arka Plandaki Sır: `try-finally` ve `Dispose()`

C# derleyicisi `using` bloğunu otomatik olarak şu koda çevirir:

```
MySqlConnection conn = new MySqlConnection(...);
try
{
    conn.Open();
    // Riskli SQL işlemleri...
}
finally
{
    // Hata çıksa da çıkmasa da KESİNLİKLE kapanır!
    conn.Dispose();
}
```

Modern C# Yazımı: `using var` Bildirimi

Süslü parantez kirliliğini önleyen modern C# 8+ tek satır yazımı:

```
public bool Guncelle(Calisan c)
{
    // Süslü parantez açmaya gerek yoktur:
    using var conn = VeritabaniBaglanti.BaglantiGetir();
    using var cmd = new MySqlCommand(sql, conn);

    conn.Open();
    return cmd.ExecuteNonQuery() > 0;
    // Metot bittiği an her iki nesne de otomatik kapatılır!
}
```

13. LINQ ile Filtreleme, Sıralama ve Sorgulama (Döngü Hamallığına Son)

LINQ (Language Integrated Query), C# koleksiyonları (`List<T>`) üzerinde SQL yazıyormuş gibi filtreleme, arama ve sıralama yapmayı sağlayan güçlü bir teknolojidir.

Eski Yol: 8 Satırlık Döngü Hamallığı

```
// Teknisyenleri bulup listelemek:
List<Calisan> filtreli = new List<Calisan>();
foreach (var c in tumCalisanlar)
{
    if (c.Yetki == "Teknisyen" && c.Maas > 20000m)
    {
        filtreli.Add(c);
    }
}
```

Modern LINQ: Tek Satırda Temiz Kod

```
// Aynı iş tek bir akıcı satırda:
var filtreli = tumCalisanlar
    .Where(c => c.Yetki == "Teknisyen" && c.Maas > 20000m)
    .OrderBy(c => c.Ad)
    .ToList();
```

LINQ Metodu	SQL Karşılığı	Örnek C# Kullanımı	Açıklama / Döndürdüğü Değer
`.Where(...)`	`WHERE`	`liste.Where(c => c.AktifMi == true)`	Şarta uyan tüm elemanları süzer (filtreler).
`.OrderBy(...)`	`ORDER BY ASC`	`liste.OrderBy(c => c.Maas)`	Küçükten büyüğe sıralar (Tersi: `OrderByDescending`).
`.Select(...)`	`SELECT`	`liste.Select(c => c.Ad).ToList()`	Nesnelerin sadece istenen alanını çekip yeni liste yapar.
`.FirstOrDefault(...)`	`LIMIT 1`	`liste.FirstOrDefault(c => c.Id == 5)`	İlk eşleşeni bulur; liste boşsa çökmez, `null` döner.
`.Any(...)`	`EXISTS`	`liste.Any(c => c.Tc == "12345678901")`	Şarta uyan kayıt var mı bakar; **`true` / `false`** döner.
`.Count()` / `.Sum()`	`COUNT / SUM`	`liste.Sum(c => c.Maas)`	Eleman sayısını veya toplam maaşı hesaplar.

> 💡 **Sonda Neden `.ToList()` Yazarız? (Ertelenmiş Çalışma - Deferred Execution):**
> LINQ sorguları `.ToList()` veya `.FirstOrDefault()` gibi bir sonuç metodu çağrılana kadar bellekte işlem yapmaz; sadece bir "plan" olarak bekler. Sonuna `.ToList()` yazdığınız anda filtreleme motoru çalışır ve verileri bellekte somut bir `List<T>` haline getirir.